

Trabajo PID

Reconocimiento de Dinero Uruguayo

Repositorio Github:

<https://github.com/FedericoVareika/trabajo-pid-reconocimiento-dinero>

Guillermo Golpe, Tomás Hazan, Federico Vareika

2026

Resumen

Este trabajo presenta el desarrollo de un sistema de reconocimiento automático de billetes uruguayos basado en la extracción y correspondencia de características locales. El objetivo principal fue explorar cómo una máquina puede emular la identificación visual del dinero frente a variaciones del mundo real (desgaste, arrugas, iluminación). Para ello, se implementaron y compararon los algoritmos SIFT y ORB, elegidos como alternativas de código abierto frente a enfoques privativos como SURF. El proyecto abarcó la construcción de una base de datos propia con imágenes de referencia de las seis denominaciones actuales y un conjunto de prueba con fotografías cotidianas. La solución definitiva y más robusta consistió en la implementación de un mecanismo de umbrales adaptativos entrenados por categoría, resolviendo la asimetría natural en la generación de correspondencias entre las distintas caras de los billetes.

Palabras clave: Procesamiento de imágenes, Billetes uruguayos, SIFT, ORB, Umbrales adaptativos.

Índice

1. Introducción	4
2. Planteamiento Teórico	5
2.1. SIFT	5
2.1.1. Detección de puntos candidatos	5
2.1.2. Refinar localización de puntos clave.	7
2.1.3. Asignar orientación a los puntos clave	8
2.1.4. Generar descriptores.	9
2.2. ORB	10
2.2.1. Detección de puntos clave y Orientación (oFAST)	10
2.2.2. Descriptor de puntos clave (rBRIEF)	11
3. Comparativa: SIFT vs. ORB	12
3.1. Similitudes	12
3.2. Diferencias en los Descriptores	12
3.3. Diferencias en la Evaluación de Similitud y Métricas de Distancia	12
3.3.1. Distancia L2 (Euclidiana) para SIFT	13
3.3.2. Distancia de Hamming para ORB	13
3.4. Resumen de Comparación	13
4. Implementación	14
4.1. Generación de la Base de Datos de Características	14
4.2. Entrenamiento de Umbrales Adaptativos	14
4.3. Evaluación con Imágenes de Prueba	15
5. Experimentación	16
5.1. Construcción de la base de datos de referencia	16
5.2. Intento de segmentación de billetes de prueba en cuadrantes	16
5.3. Umbrales genéricos versus umbrales entrenados	21
5.4. Iteración experimental sobre parámetros	22
5.5. Desafíos y limitaciones observadas	23
6. Manual de Usuario	24
6.1. Requisitos del Sistema	24
6.2. Estructura de Directorios	24
6.3. Instrucciones de Ejecución	24
6.3.1. Preparación de Datos	24
6.3.2. Entrenamiento y Primera Ejecución	25
6.3.3. Evaluación de Resultados	25

6.4. Ajuste de Parámetros	25
7. Conclusión	26
8. Autoevaluaciones	27
8.1. Tomás Hazan	27
8.2. Guillermo Golpe	28
8.3. Federico Vareika	30
9. Tabla de Tiempos	32
9.1. Tomás Hazan	32
9.2. Guillermo Golpe	33
9.3. Federico Vareika	35

1. Introducción

El proyecto que hemos realizado se desarrolla para la asignatura Procesamiento de Imágenes Digitales, con el objetivo de explorar y profundizar en áreas relacionadas a la asignatura que no fueron abordadas durante el curso. En este sentido, hemos elegido trabajar sobre el reconocimiento automático de patrones sobre un objeto determinado, en nuestro caso nos hemos inclinado específicamente sobre un sistema de reconocimiento de billetes (dinero) uruguayo. Aunque para nosotros, como seres humano, identificar un billete puede ser una tarea trivial y casi instantánea, nos ha resultado un desafío técnico sumamente interesante investigar cómo una máquina puede emular este proceso, cuáles son las etapas de procesamiento necesarias para lograrlo y qué algoritmos ofrecen una mayor fiabilidad frente a las variaciones del mundo real.

Nuestra investigación se centra en entender e implementar los mecanismos que permiten a un sistema computacional “comprender” la identidad de un billete a través de sus características visuales. Para ello, tomamos como punto de partida el artículo científico *Robust and Effective Component-Based Banknote Recognition for the Blind*. Si bien este trabajo académico nos sirvió de base para comprender la lógica de la extracción de características, decidimos adaptar y evolucionar su enfoque. Una de las primeras decisiones técnicas que tomamos fue la sustitución del algoritmo SURF (propuesto en el paper original) por alternativas de código abierto. Dado que SURF es un algoritmo bajo licencia comercial y con restricciones de uso, optamos por investigar y aplicar los algoritmos SIFT (Scale-Invariant Feature Transform) y ORB (Oriented FAST and Rotated BRIEF), disponibles en la biblioteca OpenCV.

El motor desarrollado nos permite comparar y sacar conclusiones acerca del desempeño de estos dos descriptores, evaluando no solo su precisión, sino también su comportamiento ante distintos niveles de complejidad, todo esto se verá y será analizado a lo largo de esta documentación. Para llevar a cabo estas pruebas de forma rigurosa, nos propusimos construir nuestra propia base de datos de billetes uruguayos. Este proceso fue fundamental para los algoritmos seleccionados, ya que necesitábamos dos tipos de conjuntos para poder trabajar: imágenes modelo por un lado, capturadas en condiciones ideales de iluminación y alineación para la referencia, e imágenes de prueba que simulan el uso diario del dinero. Estas últimas incluyen billetes arrugados, desgastados, con ruido visual, etc, lo que nos proporcionó un escenario real para medir la efectividad de los procesos de normalización y emparejamiento de características.

A lo largo de este informe, detallaremos cómo transformamos los planteos iniciales en un sistema funcional, analizando cada recurso utilizado, los desafíos técnicos encontrados junto con sus respectivas soluciones, y los conocimientos adquiridos durante el proceso de investigación y desarrollo del motor de reconocimiento.

2. Planteamiento Teórico

2.1. SIFT

SIFT (Scale Invariant Feature Transform)[4] es un algoritmo de detección de características locales en imágenes, que presenta estabilidad frente a transformaciones de escala y rotación.

Este algoritmo, a pesar de ser computacionalmente costoso, se ha demostrado útil para múltiples aplicaciones; por ejemplo el reconocimiento de objetos[4]. Su implementación básica describe 4 etapas necesarias:

- Detección de puntos candidatos.
- Refinar localización de puntos clave.
- Asignar orientación a los puntos clave.
- Generar descriptores de los puntos clave.

2.1.1. Detección de puntos candidatos

Para obtener los puntos candidatos, se buscan puntos que se suelen identificar bajo múltiples visualizaciones de la misma imagen. Es decir, puntos que se repiten a lo largo de todas las variaciones de escala aplicadas.

Las escalas se aplican con el filtro gaussiano, y luego se identifican los puntos utilizando la diferencia de gaussianos (DoG). Dada la función del espacio de escala de una imagen $L(x, y, \sigma)$, podemos definir la diferencia de gaussianos $D(x, y, \sigma)$:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \quad (1)$$

Siendo k un factor multiplicativo constante. Este proceso de aplicar un filtro gaussiano se repite $s + 3$ veces, siendo s una constante al igual que k y le damos a $k = 2^{1/s}$. Por ende tenemos:

$$D_i(x, y, k^i \sigma) = D_i(x, y, 2^{i/s} \sigma) \quad \forall i \in [0, s + 3] \quad (2)$$

Con estas imágenes filtradas, calculamos $s + 2$ diferencias que usamos para el siguiente paso. A esto le llamamos un octavo.

Luego tomamos la imagen filtrada definida por $D_s(x, y, 2\sigma)$ (doble σ), le hacemos un ‘down-sample’ con factor de 2, y realizamos otro octavo con esta imagen como D_0 . Este proceso lo podemos ver en 1.

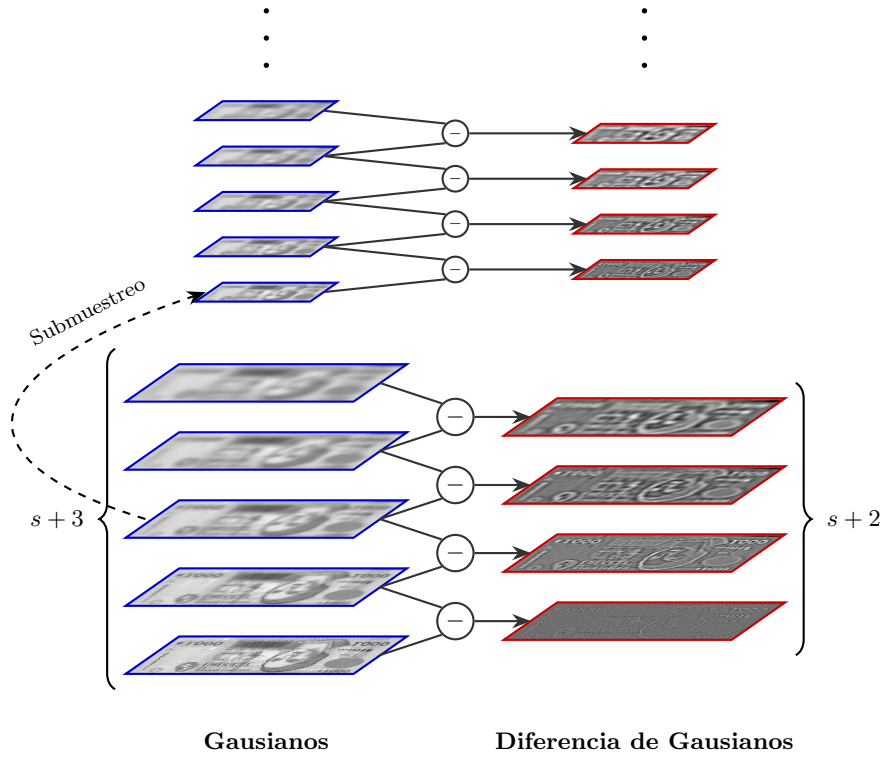


Figura 1: Demostración del comportamiento del filtro gaussiano y la diferencia de gaussianos con un billete de mil pesos. Se usa $S = 2$ como ejemplo.

Una vez que tenemos las diferencias de gaussianos, obtenemos los puntos clave que son máximos o mínimos locales. Comparamos todos los puntos con sus 26 vecinos, y si este es máximo o mínimo, entonces lo usamos para la siguiente etapa. Este procedimiento se puede ver en la figura 2.

Los valores de s y σ se experimentaron en [4] y se llegó a la conclusión que $s = 3$ y $\sigma = 1,6$ son los mejores en términos de repetibilidad de resultados, y son los que los que usa OpenCV ([1]) por defecto.

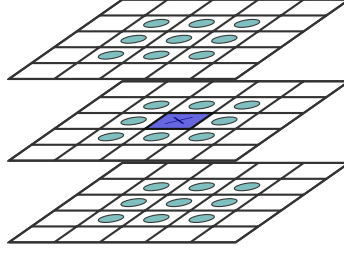


Figura 2: Diagrama del 3x3x3 armado para comparar un punto de interés con sus 26 vecinos. El punto de interés es la 'X' en azul, y sus vecinos son los círculos celestes. Debemos aclarar que este diagrama es sumamente similar a la Figura 2 de [4] ya que lo tomamos como referencia directa.

2.1.2. Refinar localización de puntos clave.

La etapa anterior devuelve demasiados puntos de interes, y en esta etapa se busca limpiarlos. Se eliminan los puntos que tienen bajo contraste y los que caen sobre un borde.

Para esto, el artículo de Lowe[4] primero interpola los puntos con un algoritmo desarrollado por Brown[2] que utiliza la serie de Taylor de la función $D(x, y, \sigma)$ ¹. Esto sirve para localizar los punto de interés con mayor precisión (escala y espacio).

Luego se chequea el valor de la función D en los nuevos extremos localizados, si este es menor a 0,03, el punto se descarta por tener contraste insuficiente.

Para eliminar los puntos localizados sobre un borde, se comparan las dos curvaturas principales en el punto. Si una es mucho mas alta que su perpendicular, se descarta ya que se categoriza como punto sobre un borde. Podemos ver ejemplos en la figura 3.

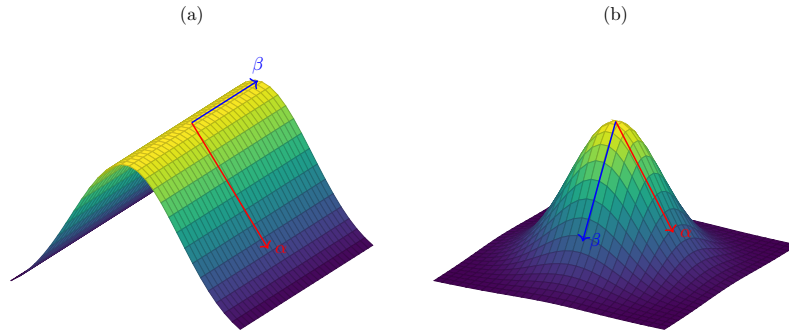


Figura 3: (a): La curvatura $\alpha \gg \beta$, por ende es un borde. (b): La curvatura $\alpha \approx \beta$, por ende no es un borde.

¹Los detalles de las ecuaciones utilizadas van por fuera de la investigación

2.1.3. Asignar orientación a los puntos clave

Para darle orientación a un punto clave, primero se ubica la imagen filtrada con la escala σ más cercana a la del punto. Luego se toma una muestra de píxeles cercanos a las coordenadas del punto, y se calcula el gradiente (escala y ángulo) de cada pixel con sus 4 vecinos (norte, este, sur, oeste). Estos gradientes son escalados según su cercanía al punto clave original, y se agregan a un histograma con 36 contenedores (uno cada 10°).

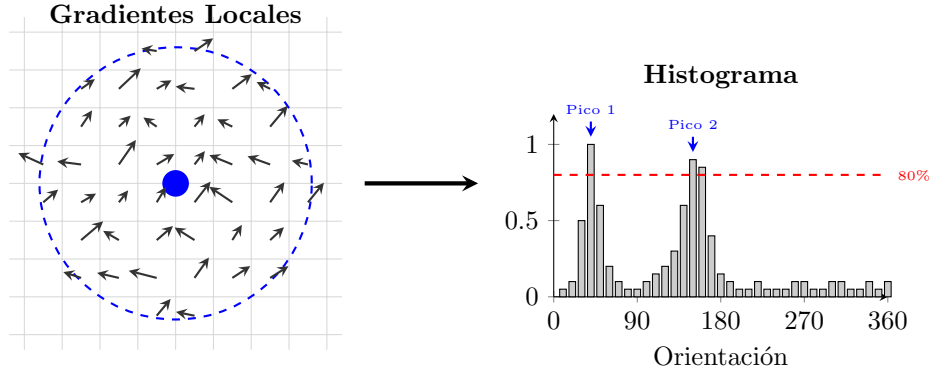


Figura 4: Creación del histograma de gradientes locales. Acá se ven 2 picos válidos, por ende de este punto clave salen 2 nuevos; uno con orientación 45° y otro con orientación 160° , ambos con misma escala y coordenadas.

Se obtiene el pico en el histograma, y se obtiene una estimación de la orientación según el pico y sus 2 vecinos. Esta orientación es la que se le asigna al punto clave. Aparte de esto, los contenedores con más de 80% del pico también se toman en cuenta para las orientaciones, por ende se generan copias de los puntos clave con las orientaciones de estos 4.

2.1.4. Generar descriptores.

Ahora que tenemos las coordenadas, la escala, y la orientación de los puntos clave, tenemos que generar los descriptores para *describir* que hay alrededor de cada punto clave.

Para crear un descriptor a partir de un punto clave, tomamos muestras de gradientes alrededor del descriptor de la imagen con filtrado correspondiente a la escala del punto. Como podemos ver en la figura 5 estas muestras son tomadas según la orientación del punto clave. En la figura se ve un muestreo de 8×8 , pero en realidad se toma un muestreo de 16×16 [4]. Estas muestras son ajustadas según su cercanía al punto, y agrupadas en histogramas. En la figura es un 2×2 de histogramas, pero en realidad se toma un 4×4 .

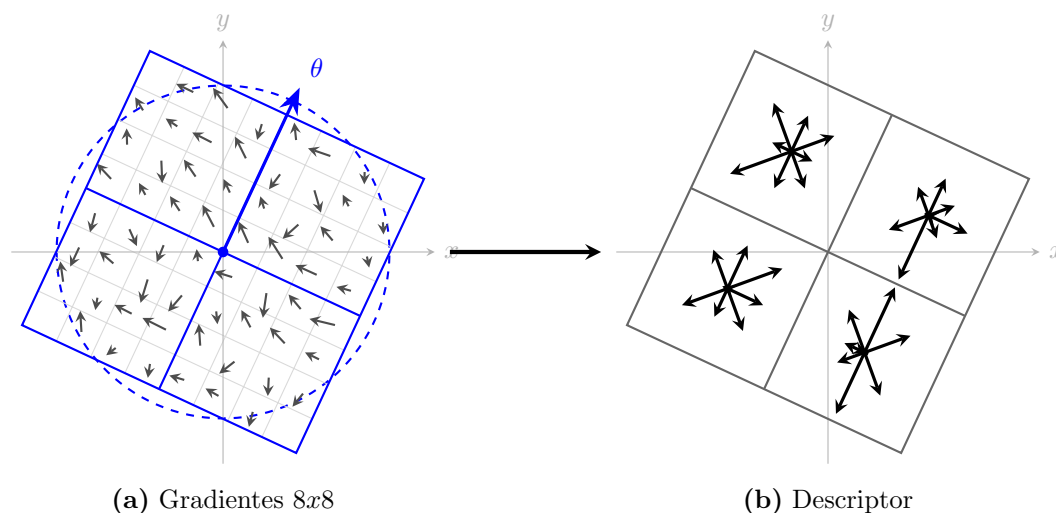


Figura 5: Creación del descriptor a partir de un punto clave con orientación θ . Se toma un 8×8 de muestras alrededor del punto, y se resumen en 4 histogramas de 8 contenedores formando una estructura 2×2 . Debemos aclarar que este diagrama es sumamente similar a la Figura 7 de [4] ya que lo tomamos como referencia directa, pero queremos mostrar cómo la orientación del punto clave afecta al descriptor.

Después de obtener los histogramas, codificamos los valores a un vector de números (float 32 bits). Como tenemos $4 \times 4 = 16$ histogramas, cada uno con 8 contenedores, la dimensión de los vectores es de $16 \times 8 = 128$ elementos ($128 \times 32 = 4096$ bits). Este vector sufre algunas transformaciones, dentro de ellas una normalización para que distintas características de la imagen (por ejemplo el contraste) no tengan efecto en el valor del descriptor. Esto se hace para todos los puntos clave, obteniendo una lista de descriptores, finalizando el algoritmo.

2.2. ORB

ORB (Oriented FAST and Rotated BRIEF)[5] es un algoritmo de detección y descripción de características locales diseñado como una alternativa eficiente a SIFT y SURF. Su principal ventaja es que es computacionalmente liviano, lo que lo hace ideal para dispositivos de baja potencia o aplicaciones en tiempo real[5]. Además, a diferencia de otros descriptores, ORB no tiene restricciones de licencia.

Su implementación se basa en la combinación del detector de puntos clave FAST y el descriptor BRIEF, introduciendo modificaciones para lograr invariancia a la rotación y robustez frente al ruido. El proceso describe 4 etapas fundamentales:

- Detección de puntos clave mediante FAST en una pirámide de escalas.
- Filtrado y refinamiento de puntos utilizando la medida de Harris.
- Asignación de orientación mediante el centroide de intensidad (oFAST).
- Generación de descriptores binarios con BRIEF rotado (rBRIEF).

2.2.1. Detección de puntos clave y Orientación (oFAST)

Para la detección inicial, se utiliza el algoritmo FAST-9 (radio circular de 9 píxeles)[5]. Dado que FAST no produce características multiescala de forma nativa, se construye una pirámide de imágenes (similar a la pirámide en SIFT) y se detectan puntos en cada nivel.

Para mitigar la alta respuesta de FAST en bordes, se aplica un filtro de Harris para ordenar los puntos y conservar los N mejores candidatos.

Para darle orientación a los puntos, se emplea el método del centroide de intensidad. Se asume que la intensidad de una esquina está desplazada de su centro, y este vector se usa para calcular la orientación. Definimos los momentos de un parche como:

$$m_{pq} = \sum_{x,y} x^p y^q I(x, y) \quad (3)$$

Siendo que x e y son relativos al punto O , encontrado usando FAST, y cubren el parche definido por el radio r . Con estos momentos, se calcula el centroide C y la orientación θ del parche mediante las funciones:

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right) \quad (4)$$

$$\theta = \text{atan2}(m_{01}, m_{10}) \quad (5)$$

Este enfoque resulta significativamente más estable que los métodos basados en histogramas de gradientes bajo condiciones de ruido intenso[5].

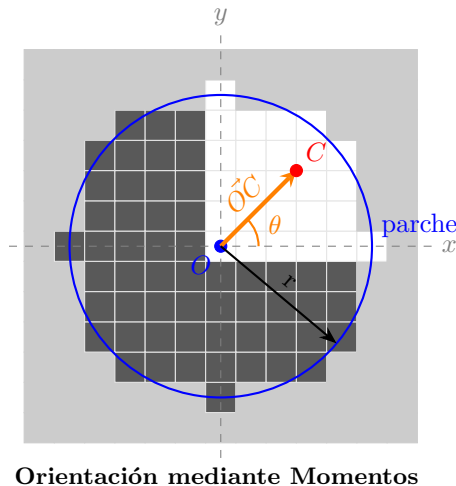


Figura 6: Se obtiene el ángulo θ usando el centroide C calculado con los valores de intensidad dentro del parche (de radio r , y centro O).

2.2.2. Descriptor de puntos clave (rBRIEF)

ORB utiliza una evolución del descriptor BRIEF, el cual genera una cadena de bits mediante pruebas binarias de intensidad entre pares de píxeles en un parche suavizado p [5]. Una prueba binaria τ se define como:

$$\tau(p; x, y) := \begin{cases} 1 & : p(x) < p(y) \\ 0 & : p(x) \geq p(y) \end{cases} \quad (6)$$

Para lograr la invariancia a la rotación, se define la matriz de pruebas S y se construye una versión "dirigida" (steered) S_θ utilizando la matriz de rotación R_θ correspondiente a la orientación calculada previamente:

$$S_\theta = R_\theta S \quad (7)$$

Sin embargo, la rotación directa de BRIEF provoca una pérdida de varianza y un aumento en la correlación de las pruebas, lo que reduce su capacidad de distinción. Para compensar esto, ORB utiliza un algoritmo de aprendizaje (rBRIEF), que maximiza la utilidad de cada bit en el descriptor final.

Usando este algoritmo, nos quedamos con descriptores de 256 bits (32 bytes).

3. Comparativa: SIFT vs. ORB

A lo largo de la investigación y el desarrollo del motor de reconocimiento, se han analizado en profundidad los algoritmos SIFT y ORB. Aunque ambos comparten el objetivo de extraer características locales robustas, presentan diferencias fundamentales en su arquitectura, eficiencia y representación de datos.

3.1. Similitudes

Ambos algoritmos buscan la invariancia ante transformaciones geométricas comunes en el mundo real, como la rotación y el cambio de escala. Para lograr la invariancia a la escala, tanto SIFT como ORB emplean una estructura de pirámide de imágenes:

- **SIFT:** Utiliza la Diferencia de Gaussianos (DoG) sobre octavas de la imagen para identificar puntos candidatos.
- **ORB:** Construye una pirámide de escalas y aplica el detector FAST en cada nivel para compensar la falta de multiescala nativa de dicho detector.

3.2. Diferencias en los Descriptores

La distinción más significativa radica en la naturaleza del descriptor generado para cada punto clave:

- **Descriptores de SIFT:** Genera un vector de 128 dimensiones de números de punto flotante (32 bits). Este vector representa un histograma de las orientaciones de los gradientes locales en una región de 16×16 píxeles, lo que resulta en un descriptor denso de 4096 bits.
- **Descriptores de ORB:** Emplea descriptores binarios basados en una evolución de BRIEF (rBRIEF). El resultado es una cadena de 256 bits (32 bytes), lo que lo hace significativamente más compacto y ligero de procesar que el de SIFT.

3.3. Diferencias en la Evaluación de Similitud y Métricas de Distancia

Debido a la diferencia en la representación de los descriptores, el cálculo de la similitud entre dos puntos (matching) requiere métricas matemáticas distintas en la implementación:

3.3.1. Distancia L2 (Euclidiana) para SIFT

Para los descriptores de SIFT, se utiliza la norma L_2 mediante `cv2.NORM_L2`. Al ser vectores de valores reales, la similitud se mide calculando la distancia euclidiana entre ellos en un espacio de 128 dimensiones:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (8)$$

Este proceso es computacionalmente costoso, lo que contribuye a que SIFT sea más lento en aplicaciones en tiempo real.

3.3.2. Distancia de Hamming para ORB

Para los descriptores binarios de ORB, se emplea la distancia de Hamming mediante `cv2.NORM_HAMMING`. En lugar de cálculos aritméticos complejos, la distancia se determina contando el número de posiciones en las que los bits de las dos cadenas son diferentes utilizando la operación XOR. Esta métrica es extremadamente eficiente en hardware.

3.4. Resumen de Comparación

Característica	SIFT	ORB
Tipo de Descriptor	Vector Real (128-d)	Vector Binario (256-bit)
Tamaño (bits)	4096	256
Métrica de Similitud	Distancia L_2	Distancia Hamming
Invariancia	Escala y Rotación	Escala y Rotación
Costo Computacional	Alto	Muy Bajo

Cuadro 1: Diferencias técnicas clave entre SIFT y ORB en la implementación del motor.

4. Implementación

4.1. Generación de la Base de Datos de Características

El sistema construye una base de datos de referencia para permitir una comparación eficiente durante la ejecución. Este proceso se centraliza en la función `get_db_features` del script de implementación.

- **Preprocesamiento:** Las imágenes originales se normalizan para que su dimensión máxima (ancho o alto) sea de 1024 píxeles. Posteriormente, se convierten a escala de grises ya que SIFT y ORB trabajan sobre intensidad y no color.
- **Extracción y Persistencia:** Se utiliza la función `extract_features` para obtener los *keypoints* y descriptores mediante los algoritmos SIFT u ORB. Los descriptores se almacenan en archivos binarios de NumPy (`.npy`) dentro de los directorios `sift_database` u `orb_database`.
- **Configuración de Algoritmos:** Hemos definido un limite ambos para SIFT y ORB de 2500 características.

4.2. Entrenamiento de Umbrales Adaptativos

La idea de umbrales adaptativos la tomamos del artículo de referencia [3].

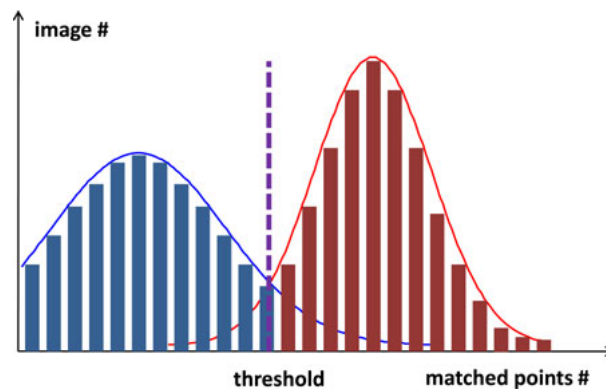


Figura 7: Gráfico mostrando los histogramas de matches positivos (rojo) y negativos (azul). Este gráfico es tomado del artículo de referencia [3].

Para cada billete de referencia, se calculan dos distribuciones de *matches*: una para imágenes de la misma categoría (positivas) y otra para denominaciones distintas (negativas), y se cargan en dos histogramas.

La función `find_intersection` calcula el umbral mediante la intersección de los histogramas de densidad de ambas distribuciones. Estos histogramas son suavizados previamente con una convolución de media móvil de tamaño 3 para estabilizar los resultados. Los valores resultantes se exportan a un archivo `thresholds.json`.

4.3. Evaluación con Imágenes de Prueba

La fase de evaluación (`evaluate`) prueba el modelo con billetes arrugados, desgastados o con variaciones de iluminación.

- **Filtrado de Correspondencias:** Se utiliza el *Ratio Test* de Lowe con un parámetro de 0,7 (`RATIO_THRESH`). Este método descarta *matches* ambiguos comparando la distancia del mejor candidato frente al segundo mejor.
- **Métricas de Distancia:** El sistema adapta la comparación según el algoritmo: para SIFT se emplea la norma L_2 (distancia euclidiana) sobre vectores de 128 dimensiones, mientras que para ORB se utiliza la norma de Hamming sobre descriptores binarios de 256 bits.
- **Consolidación de Resultados:** El sistema realiza la predicción basándose en un *scoreboard* que acumula los *matches* que superan el umbral entrenado para cada denominación. Al evaluar, se eliminan los sufijos "Polimero" para agrupar las distintas variantes de billetes de \$20 y \$50 en sus categorías reales.

5. Experimentación

5.1. Construcción de la base de datos de referencia

El primer paso del proceso experimental consistió en construir la base de datos con la que el sistema compara las imágenes de prueba. Las imágenes originales de los billetes, obtenidas de fuentes digitales en formatos variados (WebP, JPEG), fueron convertidas a formato PNG y sometidas a un proceso de normalización que estandariza sus dimensiones y condiciones de contraste. El sistema contempla ambas caras de todas las denominaciones disponibles: 20, 50, 100, 200, 500 y 1000 pesos uruguayos, incluyendo las variantes en polímero de los billetes de 20 y 50 pesos.

Sobre las imágenes normalizadas, se utilizó la herramienta interactiva de OpenCV (`selectROIs`) para que un operador humano seleccionara manualmente entre 3 y 6 regiones de interés por cada cara de cada denominación. Las regiones seleccionadas abarcan elementos como retratos, denominaciones numéricas, escudos y patrones de seguridad; es decir, zonas que presentan diferencias visuales marcadas entre denominaciones y que, por lo tanto, generan descriptores altamente diferenciadores. Una vez recortadas, cada región fue procesada por SIFT y ORB para extraer sus descriptores, que se almacenaron como archivos NumPy (`.npy`) junto con imágenes de visualización que muestran los keypoints detectados.

5.2. Intento de segmentación de billetes de prueba en cuadrantes

Como parte de la experimentación, se evaluó una técnica alternativa de preprocesamiento que consistía en segmentar las imágenes de prueba en cuatro cuadrantes dispuestos en una grilla de 2×2 , con el fin de localizar espacialmente las correspondencias y reducir la influencia de patrones compartidos entre denominaciones en zonas no discriminativas.

Módulo de recorte automático (*auto-crop*): Antes de poder dividir el billete en cuadrantes, era necesario aislar el billete del fondo de la imagen. Cuando un billete es fotografiado en condiciones cotidianas, la imagen incluye el fondo sobre el que está apoyado (una mesa, una mano, una hoja), lo que introduce regiones completamente irrelevantes para el *matching*. Si se dividiera la imagen cruda en cuatro cuadrantes, los cuadrantes no corresponderían a zonas del billete sino a una mezcla arbitraria de billete y fondo, volviendo el enfoque directamente inviable.

Para resolver esto, se implementó un módulo de recorte automático de dos etapas. En la **primera etapa**, se convierte la imagen a escala de grises y se aplica un suavizado gaussiano (kernel 15×15) para atenuar el ruido de alta frecuencia. Sobre la imagen suavizada, el detector de bordes Canny identifica las transicio-

nes de intensidad que delimitan el contorno exterior del billete. A continuación, se aplica una operación morfológica de cierre (*MORPH_CLOSE*) con un elemento estructurante rectangular de 15×15 píxeles, cuya función es unir los segmentos de borde discontinuos y cerrar los huecos que el operador de Canny puede dejar en zonas de bajo contraste, como los bordes del billete sobre un fondo claro. Del conjunto de contornos externos obtenido se selecciona el de mayor área —que corresponde al perímetro del billete— y se extrae su rectángulo delimitador alineado con los ejes. Un margen de seguridad (*trim*) de 15 píxeles se aplica simétricamente para excluir el borde difuso que el suavizado introduce alrededor del contorno real, produciendo el recorte final.

En la **segunda etapa**, que se incorporó para mejorar la calidad de los cuadrantes en casos donde el billete no estaba perfectamente paralelo al plano de la cámara, se intentó una corrección de perspectiva. Este módulo generaba tres mapas de bordes en paralelo con distintas estrategias: Canny con umbrales adaptativos basados en la mediana de intensidad, umbralización adaptativa gaussiana, y filtro bilateral seguido de Canny con umbrales bajos. Sobre cada mapa se buscaba el contorno de mayor área con exactamente cuatro vértices, empleando aproximación poligonal con distintos valores de $\varepsilon \in \{0,02, 0,03, 0,05\}$. De encontrarse un cuadrilátero válido, se aplicaba una transformación de perspectiva (*warpPerspective*) para obtener una vista frontal y rectificada del billete; en caso contrario, el algoritmo recurría a un modo de respaldo basado en el rectángulo rotado de área mínima.

Pipeline implementado: La implementación incluyó dos etapas diferenciadas. En primer lugar, un paso de recorte automático (*auto-crop*) del billete mediante detección de contornos, para aislar el billete del fondo antes de la división. Este módulo generaba tres mapas de bordes en paralelo con distintas estrategias: Canny con umbrales adaptativos basados en la mediana de intensidad, umbralización adaptativa gaussiana, y filtro bilateral seguido de Canny con umbrales bajos. Sobre cada mapa se buscaba el contorno de mayor área con exactamente cuatro vértices, empleando aproximación poligonal con distintos valores de $\varepsilon \in \{0,02, 0,03, 0,05\}$. De encontrarse un cuadrilátero válido, se aplicaba una transformación de perspectiva para obtener una vista frontal y rectificada del billete; en caso contrario, el algoritmo recurría a un modo de respaldo basado en el rectángulo rotado de área mínima. En segundo lugar, el billete recortado se dividía geométricamente en cuatro cuadrantes iguales, etiquetados Q_0 (superior-izquierdo), Q_1 (superior-derecho), Q_2 (inferior-izquierdo) y Q_3 (inferior-derecho), que se emparejaban individualmente contra los cuadrantes equivalentes de las imágenes de referencia.

Motivación: La hipótesis subyacente era que los billetes poseen zonas con funciones visuales distintas: el retrato principal suele ocupar la mitad izquierda del anverso, mientras que la denominación numérica y los elementos de seguridad se concentran en la mitad derecha. Al comparar por cuadrante, se esperaba ob-

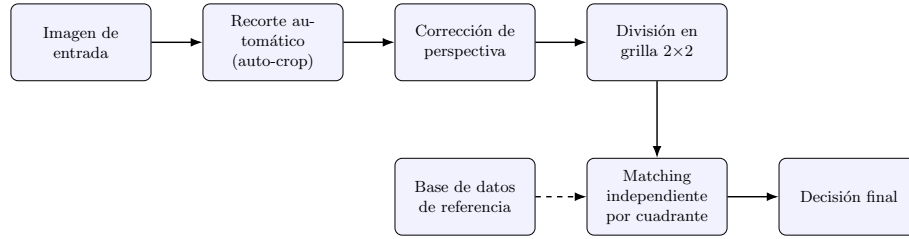


Figura 8: Diagrama de bloques del pipeline de segmentación en cuadrantes. La imagen de entrada se recorta automáticamente y se corrige la perspectiva; luego se divide en una grilla 2×2 y cada cuadrante se compara de forma independiente contra la base de datos de referencia.

tener correspondencias más específicas y con menor probabilidad de cruce entre denominaciones que comparten paletas de color o patrones similares.

Por qué no implementar cuadrantes en la BD: La segmentación en cuadrantes de las imágenes de referencia parte de una premisa incorrecta: que una rejilla geométrica fija es capaz de aislar regiones semánticamente coherentes de un billete. En la práctica, el corte al centro de la imagen es completamente ciego al contenido y puede partir exactamente por la mitad el retrato, los números de denominación o cualquier otro elemento distintivo, dejando cada cuadrante con una característica incompleta que ningún descriptor puede representar ni reconocer de forma fiable. A esto se suma que la comparación cuadrante-a-cuadrante exige una alineación casi perfecta entre la imagen de prueba y la de referencia: una pequeña diferencia de rotación o escala (inevitable en cualquier fotografía real) hace que el Q1 de una imagen corresponda en realidad a una mezcla del Q1 y Q2 de la otra, rompiendo por completo la correspondencia esperada. Finalmente, dividir la imagen en cuatro partes no aporta información nueva respecto a usar el billete completo, simplemente fragmenta los mismos descriptores e introduce fronteras artificiales como fuente de error adicional. Teniendo en cuenta que el matching sobre imagen completa ya resuelve de forma natural todos estos problemas (permitiendo que los descriptores se busquen entre sí sin restricciones geométricas), el enfoque por cuadrantes no solo era inferior, sino que sus limitaciones eran predecibles a priori sin necesidad de experimentación.

Resultados y análisis de fallos: Los resultados de este enfoque no mostraron mejoras respecto al análisis del billete completo, y en varios casos degradaron la precisión. Se identificaron cuatro causas principales:

1. **Truncamiento de elementos semánticos:** La división geométrica rígida no respeta las fronteras semánticas del billete: un retrato o una denominación numérica puede quedar partido entre dos cuadrantes, perdiendo su valor

informativo como elemento unitario. Un billete con relación de aspecto aproximada 2:1 dividido en una grilla 2×2 produce cuatro regiones de relación 1:1, que sistemáticamente cortan el retrato central cuando el billete no está perfectamente centrado en la imagen.



Figura 9: Grilla 2×2 superpuesta sobre el anverso de un billete de \$100. Las líneas de corte geométrico atraviesan el retrato central, fragmentando el elemento más discriminativo entre los cuadrantes Q_0 y Q_1 .

2. **Escasez de keypoints por cuadrante.** Al reducir cada región a una cuarta parte del área total, la cantidad de keypoints detectables disminuye proporcionalmente. En cuadrantes con texturas homogéneas el número de correspondencias puede caer por debajo del umbral mínimo incluso para la denominación correcta, haciendo la comparación no confiable o directamente inviable.
3. **Fragilidad del recorte automático.** El recorte automático presentó limitaciones con billetes arrugados o doblados, donde el contorno visible no es rectangular. En esos casos, ninguna de las tres estrategias de detección de bordes encontraba un cuadrilátero válido, y el modo de respaldo producía recortes con deformaciones residuales que se propagaban como error sistemático a la etapa de división posterior.
4. **Asimetría entre base de datos y consulta.** La base de datos de referencia fue construida con regiones semánticas seleccionadas manualmente (un retrato completo, un número entero) que no se corresponden con los cuadrantes geométricos, creando una asimetría fundamental entre las representaciones de referencia y de consulta. Mientras las referencias codifican semántica, las consultas codifican geometría.



Figura 10: Ejemplo de fallo en el recorte automático. (Izquierda) Imagen de entrada con el billete ligeramente doblado. (Derecha) Resultado del recorte con deformaciones residuales, que se propagan directamente a la división en cuadrantes.

(Izquierda) Referencia

Componente semántico



Retrato completo
recortado manualmente

(Derecha) Consulta

Cuadrante geométrico Q_0



Cuadrante con fondo
decorativo sin retrato

Figura 11: Asimetría entre representaciones. (Izquierda) Componente semántico de referencia: retrato del billete de \$100 recortado manualmente. (Derecha) Cuadrante Q_0 geométrico del mismo billete, que incluye fondo decorativo y excluye el retrato.

Este experimento condujo a un cambio de enfoque: en lugar de segmentar las imágenes de prueba, se optó por mejorar el mecanismo de decisión mediante el entrenamiento de umbrales adaptativos por categoría. Para cada denominación y componente semántico de referencia se calculó el histograma de puntuaciones de *matching* tanto para imágenes positivas (misma denominación) como negativas (denominaciones distintas), determinando el umbral óptimo como el punto de intersección entre ambas distribuciones. Este proceso demostró ser una solución más efectiva al problema de la discriminación entre denominaciones, ya que opera sobre el billete completo y respeta la estructura semántica de la base de datos.

5.3. Umbrales genéricos versus umbrales entrenados

La primera versión funcional del sistema utilizaba un umbral fijo y universal de `MIN_MATCHES = 10` para decidir si una correspondencia entre la imagen de prueba y un elemento de la base de datos era válida. Este valor se justifica teóricamente por ser más del doble del mínimo geométrico necesario para estimar una homografía (4 puntos). Sin embargo, la experimentación reveló rápidamente que un umbral único resulta insuficiente para un sistema que debe discriminar entre múltiples denominaciones con patrones compartidos.

El problema fundamental es que las distintas caras de los distintos billetes no generan la misma cantidad de correspondencias válidas ante imágenes de su misma denominación. Por ejemplo, el reverso de un billete de 500 pesos, que contiene la fachada del Palacio Legislativo con una gran riqueza de detalles arquitectónicos, puede generar más de 200 correspondencias válidas con una fotografía real de ese billete. En cambio, el frente de un billete de 1000 pesos, con grandes áreas lisas y pocas texturas, podría generar apenas 30 correspondencias con una foto legítima de ese mismo billete. Si se utiliza un umbral fijo de 10, ambas caras aceptarían la correspondencia, pero también lo harían muchas imágenes de otras denominaciones que casualmente alcancen las 10 correspondencias, generando falsos positivos. Si se sube el umbral a 50 para evitar estos falsos positivos, el billete de 1000 dejaría de ser reconocido en muchos casos, generando falsos negativos.

Esta asimetría se observó experimentalmente al probar con el dataset extendido de fotografías reales. Billetes fotografiados con fondos complejos, variaciones de iluminación y distintos ángulos generaban distribuciones de matches muy diferentes entre denominaciones. La conclusión fue clara: cada cara de cada billete necesita su propio umbral, calibrado en función de la cantidad de correspondencias que típicamente produce con imágenes legítimas y, crucialmente, de la cantidad de correspondencias espurias que genera con imágenes de otras denominaciones.

Para resolver este problema, se implementó un mecanismo de entrenamiento de umbrales por categoría. El dataset de imágenes de prueba se divide aleatoriamente en un conjunto de entrenamiento (75 %) y un conjunto de evaluación

(25 %) mediante una semilla fija (`random.seed(42)`) para garantizar reproducibilidad. Durante la fase de entrenamiento, cada imagen del conjunto se compara contra todos los elementos de la base de datos de referencia, y para cada cara de cada billete de referencia se recopilan dos distribuciones: los matches positivos (correspondencias con imágenes de la misma denominación) y los matches negativos (correspondencias con imágenes de otras denominaciones). El umbral óptimo se calcula buscando el punto de intersección entre los histogramas de densidad de ambas distribuciones, suavizados mediante una convolución con una ventana de promedio móvil de tamaño 3. Si las distribuciones no se superponen (caso ideal en que todas las imágenes correctas generan más matches que cualquier imagen incorrecta), el umbral se fija en el punto medio entre el máximo negativo y el mínimo positivo. Los umbrales resultantes se almacenan en un archivo `thresholds.json` dentro de cada directorio de base de datos, evitando recalcularlos en ejecuciones posteriores.

Los resultados de este enfoque son reveladores. Como se muestra en la Tabla 2, los umbrales entrenados presentan una dispersión que hace inviable cualquier valor fijo alternativo.

	SIFT	ORB
Umbral mínimo (frente billete \$1000)	33	18
Umbral máximo (frente polímero \$50)	345	153

Cuadro 2: Rango de umbrales entrenados por algoritmo. La diferencia entre el valor mínimo y máximo dentro de un mismo algoritmo ilustra por qué un umbral genérico resulta fundamentalmente inadecuado.

5.4. Iteración experimental sobre parámetros

El desarrollo del sistema no fue un proceso lineal, sino que requirió múltiples ciclos de experimentación iterativa donde se ajustaron los parámetros del pipeline para observar su impacto en la precisión. El valor del Ratio Test de Lowe se probó en un rango que abarcó desde 0.6 hasta 0.85. Valores muy estrictos (0.6) descartaban demasiadas correspondencias legítimas, particularmente en fotografías tomadas con poca resolución o con desenfoco por movimiento, donde los descriptores pierden precisión y las distancias al mejor y segundo mejor candidato se acercan. Valores muy permisivos (0.85) permitían el paso de correspondencias en zonas repetitivas de los billetes (tramas de seguridad, líneas paralelas), incrementando el ruido en el sistema de votación. El valor final de 0.7 se seleccionó empíricamente como el que producía el mejor equilibrio entre retención de correspondencias verdaderas y rechazo de ruido sobre el conjunto de imágenes de prueba.

De manera análoga, el número de características extraídas por ORB ($n_{features}$) se evaluó en distintos niveles. En la versión por regiones se utilizaban 500 características por región ($5 \text{ regiones} \times 500 = 2500$ totales), mientras que en la versión de billetes completos se unificó en un único parámetro de 2500 características globales. Pruebas con valores inferiores (1000, 1500) mostraron que se perdían detalles en zonas de bajo contraste del billete, mientras que valores superiores (5000) no aportaban mejoras significativas en precisión pero incrementaban notablemente el tiempo de procesamiento.

5.5. Desafíos y limitaciones observadas

La experimentación evidenció que el factor más determinante en el rendimiento del sistema es la brecha entre las condiciones de la base de datos de referencia y las condiciones reales de las fotografías de prueba. Las imágenes de referencia provienen de fuentes digitales de alta calidad, con iluminación uniforme y sin distorsiones, mientras que las fotografías de prueba presentan billetes arrugados, parcialmente doblados, con sombras proyectadas, sobre fondos de distintos colores y texturas, y capturados con cámaras de celulares con distintas resoluciones y niveles de compresión. Esta diferencia de dominio constituye una limitación intrínseca de los enfoques basados en correspondencia de características locales, independientemente de los parámetros de ajuste empleados.

Otro desafío significativo fue la existencia de variantes de un mismo billete. Los billetes de 20 y 50 pesos existen tanto en versión papel como en versión polímero, con diseños sustancialmente diferentes. El sistema debe ser capaz de reconocer ambas versiones como pertenecientes a la misma denominación durante la evaluación, mientras mantiene representaciones separadas en la base de datos para capturar las diferencias visuales de cada variante. Esto se resolvió mediante una función de normalización que, al momento de calcular las métricas de evaluación, agrupa las predicciones eliminando el sufijo *Polimero* antes de comparar con la categoría esperada.

Finalmente, el sistema enfrenta el desafío de que la cara del billete fotografiada no es conocida de antemano. La base de datos de referencia incluye representaciones del anverso y del reverso de cada denominación, por lo que el proceso de comparación debe ejecutarse contra ambas caras y consolidar los resultados. Si bien esto se resuelve implícitamente al retener la puntuación máxima entre todas las variantes de una denominación, implica un mayor costo computacional y una mayor probabilidad de correspondencias falsas entre caras de distintas denominaciones que comparten elementos decorativos similares.

6. Manual de Usuario

Este manual describe los pasos necesarios para configurar, entrenar y ejecutar el motor de reconocimiento de billetes uruguayos utilizando los algoritmos SIFT y ORB.

6.1. Requisitos del Sistema

Para ejecutar el script `./src/MatchingFullBills.ipynb`, es necesario contar con un entorno de Python 3.10+ con las siguientes bibliotecas instaladas:

- `opencv-python` (OpenCV): Para el procesamiento de imágenes y algoritmos de visión.
- `numpy`: Para el manejo de vectores de descriptores y cálculos numéricos.
- `glob`, `json`, `re`: Para la gestión de archivos y persistencia de datos.

6.2. Estructura de Directorios

El sistema espera una estructura de carpetas específica para localizar las imágenes y almacenar la base de datos de características:

- `./data/bill_processing/processed_images/`: Contiene las imágenes de referencia (imágenes "limpiasz alineadas de cada cara de los billetes).
- `./data/test_images/`: Contiene las carpetas con las imágenes de prueba, organizadas por denominación (ej. `/100/`, `/500/`).
- `./data/full_bill_database/`: Directorio donde el sistema creará automáticamente las subcarpetas `sift_database` y `orb_database` para guardar los descriptores (`.npy`) y los umbrales entrenados (`thresholds.json`).

6.3. Instrucciones de Ejecución

6.3.1. Preparación de Datos

Asegúrese de que las imágenes de referencia en `processed_images` sigan la nomenclatura esperada por el sistema (ej. `clean_100PesosFront.png`) para que la función de extracción de denominación funcione correctamente.

6.3.2. Entrenamiento y Primera Ejecución

Al ejecutar el bloque de “Ejecución Principal”, el sistema realizará las siguientes tareas de forma automática:

- **Carga del Dataset**
- **Generación de la Base de Datos**
- **Cálculo de Umbrales**
- **Predicciones y Resultados**

6.3.3. Evaluación de Resultados

- La denominación real vs. la predicción para cada imagen.
- El estado de la predicción (**CORRECTO** o **INCORRECTO**).
- La **Precisión Global (Accuracy)** final para cada algoritmo (SIFT y ORB).

6.4. Ajuste de Parámetros

Si desea modificar el comportamiento del sistema, puede ajustar las constantes al inicio del script:

- **RATIO_THRESH**: Por defecto en 0,7. Redúzcalo para ser más estricto con los *matches* o aumentelo para permitir más correspondencias.
- **nfeatures**: Definido en 2500 para ambos algoritmos. Este valor controla cuántos puntos clave se extraen de cada imagen.

7. Conclusión

El presente trabajo nos permitió construir un motor de reconocimiento de billetes uruguayos basado en la correspondencia de características locales, implementando y comparando SIFT y ORB como alternativas de código abierto al algoritmo SURF del artículo de referencia. En términos generales, los objetivos planteados se alcanzaron satisfactoriamente: el sistema identifica correctamente la denominación de billetes fotografiados en condiciones cotidianas (con arrugas, desgaste o iluminación variable) para las seis denominaciones del peso uruguayo en sus variantes papel y polímero.

Resultados y desvíos respecto a lo esperado: La principal sorpresa fue la insuficiencia del umbral genérico inicial. La experimentación reveló que cada cara de cada billete tiene una “huella de *matches*” propia, con rangos de 18–153 para ORB y 33–345 para SIFT (Tabla 2), haciendo inviable cualquier valor fijo. El mecanismo de umbrales adaptativos entrenados por categoría [3] resolvió este problema y constituyó el mayor acierto técnico. Por otro lado, el intento de segmentación en cuadrantes no produjo mejoras: la división geométrica truncó elementos semánticos clave, redujo la densidad de *keypoints* por región e introdujo una asimetría fundamental con la base de datos, construida sobre regiones semánticas.

Propuestas de mejora y extensión: El factor limitante claro del sistema actual es la brecha entre las condiciones controladas de la base de datos y la variabilidad de las fotografías reales; ampliar el conjunto de imágenes de prueba con mayor diversidad de fondos, ángulos e iluminación robustece directamente el entrenamiento de umbrales. Como extensión natural del sistema, una mejora significativa sería habilitar el reconocimiento simultáneo de múltiples billetes en una misma imagen, lo que requeriría incorporar una etapa de detección y segmentación de cada billete antes de aplicar el *pipeline* actual de forma independiente.

Desviaciones respecto a la planificación inicial: El desarrollo no fue lineal: la primera versión con umbral fijo resultó insuficiente, lo que obligó a reasignar tiempo de la fase de evaluación hacia la implementación de umbrales adaptativos y la exploración del enfoque por cuadrantes. El proyecto se completó en el período previsto, aunque con una distribución de esfuerzo distinta a la planificada.

Lecciones aprendidas: El proyecto dejó tres aprendizajes centrales. Primero, en problemas de reconocimiento multi-clase los umbrales de decisión deben calibrarse por categoría, ya que las distribuciones de similitud son inherentemente asimétricas. Segundo, la coherencia semántica entre las representaciones de referencia y las de consulta es más determinante que la granularidad de la comparación. Tercero, la experimentación sistemática tiene valor en sí misma: cada enfoque descartado (umbral fijo, segmentación geométrica) aportó información que orientó decisiones posteriores más sólidas y, en definitiva, condujo al sistema robusto actual.

8. Autoevaluaciones

8.1. Tomás Hazan

Exposición

Comprensión y dominio

Nos ubicamos en el nivel Excelente. Considero que durante la investigación y el desarrollo adquirimos un dominio profundo de los algoritmos SIFT y ORB, comprendiendo los conceptos teóricos detrás de la extracción de características. Esto nos permitió demostrar solidez y poder tomar decisiones fundamentadas a la hora de trabajar.

Exposición didáctica

Nos evaluo entre Bien y Excelente. Creo que nuestra presentación fue clara y logramos transmitir un tema técnico complejo de forma accesible y didáctica. Para ello, nos apoyamos en diapositivas ilustrativas y ejemplos visuales que hacen amena la exposición.

Integración del equipo

Nos sitúo en la categoría de Excelente. Todos los integrantes del grupo participamos activamente, tanto en las fases de desarrollo del proyecto como en la creación de la presentación y el desarrollo de la documentación. Asimismo, todos somos capaces de demostrar un conocimiento global y equitativo del proyecto.

Implementación

Objetivos

Considero que merecemos entre Bien y Excelente. La implementación alcanzó casi todos los objetivos que habíamos marcado para el proyecto. Como se fijó en un principio, implementamos un motor de reconocimiento a partir de un marco teórico, trabajamos, desarrollamos y comparamos sobre dos algoritmos distintos (únicamente debimos hacer el cambio de SURF a SIFT por una situación ya mencionada anteriormente). Además, construimos nuestra propia base de datos. Finalmente, generamos un sistema robusto, por más que debimos hacer cambios en el desarrollo ya que nuestro producto inicial no tuvo la robustez esperada.

Aspectos didácticos

Nos ubico en el nivel Excelente. El desarrollo del código se estructuró de manera muy didáctica, incluyendo muchas explicaciones en las etapas del proceso, con comentarios que justifican las tomas de decisiones de manera comprensiva.

Experimentación y conclusiones

Considero que nuestro nivel se encuentra entre Bien y Excelente. Realizamos distintas experimentaciones en el desarrollo buscando la mejor resolutive para obtener resultados satisfactorios. Podríamos haber sido superiores si generamos una

base de datos de prueba con aún más billetes. Sin embargo, al menos con los recursos disponibles, fuimos capaces de generar conclusiones, de evaluar los modelos, entender los problemas cuando los hubo y realizar las comparativas esperadas, por lo que creo que fue un muy buen trabajo.

Documentación

Contenidos

Nos situamos en el nivel Excelente. Considero que los contenidos del informe son completos y están muy bien explicados, abarcando desde el comienzo hasta la implementación final. Además, el documento mantiene una estructura bien organizada que facilita la lectura y comprensión del trabajo.

Divulgación de los contenidos

Creo que estamos entre el nivel Bien y Excelente. A pesar de ser un proyecto técnico, los contenidos se presentan de forma clara y explicativa. Integramos imágenes de cómo se desarrolla, esquemas del funcionamiento de los algoritmos y ejemplos claros para que se puedan entender los conceptos fundamentales.

Bibliografía y Recursos científicos

Me parece que estamos entre Bien y Excelente. Hicimos un uso adecuado de recursos científicos bien referenciados, tomando como núcleo de nuestra investigación el artículo académico (Robust and Effective Component-Based Banknote Recognition for the Blind) sobre reconocimiento de billetes, pudiendo adaptar a nuestro proyecto

8.2. Guillermo Golpe

Exposición

Comprensión y dominio

En este apartado nuestro trabajo fue Excelente, la complejidad de los algoritmos a utilizar requirió de un profundo análisis y entendimiento de cada paso de ellos. Esto conllevó que podamos adaptar el proyecto a nuestras necesidades y hayamos podido encontrar soluciones exitosas.

Exposición didáctica

A mi parecer, tanto en los seguimientos como en la presente memoria, hemos demostrado un nivel Excelente de exposición, permitiendo a terceros entender la complejidad del proyecto en términos accesibles. El uso de diagramas, imágenes, y el adecuado uso de conceptos hace que la exposición del trabajo pueda ser entendida incluso por aquellos que no poseen total comprensión de las bases del proyecto.

Integración del equipo

Este apartado el nivel alcanzado fue Excelente, la constante participación de todos los integrantes del grupo y el apoyo entre nosotros facilitó de desarrollo del trabajo y concluyó en un proyecto bien desarrollado.

Implementación

Objetivos

Creo que nuestra calificación debería situarse entre un Bien y un Excelente. Logramos cumplir con la inmensa mayoría de las metas que nos habíamos propuesto para el proyecto, incluso llevando algunas de ellas a un nivel superior a lo planificado. Tal como se planificó desde el inicio, creamos un motor de reconocimiento apoyado en su respectivo marco teórico y llevamos a cabo el desarrollo y la comparativa de dos algoritmos diferentes (con el detalle de haber sustituido SURF por SIFT debido a licencias). Sumado a esto, elaboramos una base de datos propia, que creamos minuciosamente con el fin de obtener los mejores resultados posibles. Para terminar, aunque la primer versión no funcionaba como era esperado, logramos entregar un sistema robusto mediante el continuo desarrollo y búsqueda de mejora de todo el equipo, con el fin de cumplir los objetivos pero no obteniendo únicamente el resultado mínimo de cada uno de ellos.

Aspectos didácticos

Para este apartado, considero justo que nos califiquemos con un Excelente. Si revisan la programación, notarán que nos enfocamos en que cualquiera pueda entender la lógica detrás del sistema. En lugar de limitarnos a escribir el código, documentamos a fondo cada fase, dejando notas claras que explican el razonamiento exacto detrás de cada decisión técnica que tomamos.

Experimentación y conclusiones

Creo que nos merecemos una calificación que ronde entre Bien y Excelente. Durante toda la fase de construcción, probamos múltiples alternativas con el objetivo de hallar la solución óptima y llegar a buen puerto. Incluso, dejando en el repositorio la otra solución disruptiva con la cual no obtuvimos buenos resultados, a modo de ejemplificación de el nivel de experimentación alcanzado. El resultado final podría haber sido aún mejor, si tuviésemos la posibilidad de obtener una mayor cantidad de imágenes de prueba. Aun así, dadas las limitaciones, logramos cumplir: analizamos los modelos a fondo, comprendimos las fallas que surgieron sobre la marcha, sacamos conclusiones sólidas y completamos las comparaciones previstas. En definitiva, considero que la labor realizada fue sumamente positiva.

Documentación

Contenidos

Gracias a la fluidez y el orden metódico del documento, cualquier persona puede seguir el hilo del trabajo sin perderse, lo que a mi criterio nos otorga un Excelente. La documentación es totalmente integral y cubre el trayecto de nuestro desarrollo. Nos preocupamos por plasmar explicaciones para cada etapa, trazando una línea clara entre el comienzo teórico y la ejecución final del sistema.

Divulgación de los contenidos

Teniendo en cuenta el esfuerzo puesto en la claridad de la entrega, nuestra valoración debería ubicarse en Excelente. Nos preocupamos por ilustrar toda la teoría dura con apoyos visuales muy concretos. Así, mediante la inclusión de esquemas operativos de los algoritmos y situaciones de ejemplo muy representativas, conseguimos que los pilares fundamentales del proyecto se entiendan a la perfección y sin ninguna necesidad de realizar un extenso estudio por fuera de la documentación del proyecto.

Bibliografía y Recursos científicos

Para esta evaluación, creo que merecemos situarnos en Bien. Logramos nutrir el proyecto basándonos en bibliografía científica rigurosa y correctamente citada. Puntualmente, utilizamos como pilar fundamental de la investigación el artículo "(Robust and Effective Component-Based Banknote Recognition for the Blind)", logrando trasladar y ajustar de manera exitosa los conceptos de ese trabajo a las necesidades de nuestro propio desarrollo. Si bien no lo creo necesario, la implementación de alguna fuente adicional a las presentadas podría haber enriquecido el proyecto, pero dado como se desarrolló este mismo considero que no fue necesario.

8.3. Federico Vareika

Exposición

Comprensión y dominio

Considero que mi desempeño en este apartado se sitúa en el nivel Excelente. Al haber asumido la responsabilidad de redactar la explicación teórica de los algoritmos SIFT y ORB, profundicé en los fundamentos matemáticos de la detección de puntos clave y la generación de descriptores. Esta base teórica fue fundamental para la toma de decisiones técnicas durante la fase de codificación.

Exposición didáctica

Me evaluó en el nivel Excelente. Mi enfoque principal fue traducir conceptos complejos, como el espacio de escala de los Gaussianos ($L(x, y, \sigma)$) o la orientación en ORB, en una narrativa técnica clara y accesible para la memoria final.

Integración del equipo

Nivel Excelente. Logramos una sinergia muy productiva donde cada integrante aportó desde su especialidad. En mi caso, trabajé con mis compañeros para asegurar que las dos ramas de desarrollo (SIFT y ORB) convergieran en un motor

único y funcional.

Implementación

Objetivos

Nivel Excelente. Fui el encargado principal de implementar el pipeline de SIFT, asegurando la correcta detección de extremos en el espacio de escala. Asimismo, desarrollé en gran parte el script final que consolida los resultados de ambos algoritmos, permitiendo que el motor sea genérico, parametrizando los algoritmos.

Aspectos didácticos

Considero que el trabajo merece un Excelente. Me esforcé por aplicar una arquitectura de código limpia en el script `MatchingFullBills.py`, utilizando tipado de datos (`typing`) y comentarios extensos que justifican el uso del Ratio Test de Lowe y la selección de normas (L_2 para SIFT y Hamming para ORB).

Experimentación y conclusiones

Nivel Bien. Experimenté con los umbrales adaptativos, ambos con billetes enteros, y segmentos de billetes. Esto nos permitió obtener resultados muy buenos en la predicción final de los billetes.

Documentación

Contenidos

Nivel Excelente. Me encargué de la documentación de la implementación, explicando paso a paso el script final.

También trabajé en la documentación de los algoritmos, teniendo que investigar de los artículos originales (SIFT [4] y ORB [5]), y entenderlos a un nivel mayor para poder explicar y visualizar con los diversos diagramas hechos a mano.

Divulgación de los contenidos

Nivel Excelente. Procuré que la documentación de la implementación guardara una relación directa con el código fuente, facilitando que cualquier desarrollador externo pueda replicar el motor de reconocimiento siguiendo las secciones del informe.

Bibliografía y Recursos científicos

Nivel Excelente. Realicé un análisis exhaustivo del artículo núcleo de nuestra investigación y los artículos originales de SIFT y ORB, citándolos cuando es necesario.

9. Tabla de Tiempos

9.1. Tomás Hazan

Actividad Realizada	Fecha	Tiempo (hs)
Seguimiento 1		8
Decisión del tema	11/3	3
Primera aproximación para entendimiento del método ORB	14/3	2
Puesta en común de la información recaudada para metodologías	15/3	1
Desarrollo de objetivos y planificación preliminar del primer seguimiento	16/3	2
Seguimiento 2		19
Busqueda de solución debido al pantentado de SURF	23/3	3
Desarrollo de módulo de extracción (ORBFeatureRegions) y control de excepciones	24/3	4.5
Desarrollo de módulo de emparejamiento (ORBMatching) e integración de validaciones	25/3	4.5
Ayuda para la correcta creación y elaboración de las imagenes de la base de datos	26/3	2
Puesta en común del equipo respecto a avances y trabajo hecho	2/4	3
Desarrollo de documento y transofmración a latex para el segundo seguimiento	5/4	2
Seguimiento 3		13
Análisis más exhaustivo de parámetros, metodos y comparaciones	20/4	3
Prueba detallada de la base de datos con completitud de imagenes seleccionadas	21/4	3
Comienzo de documentación para la realización de la memoria	22/4	3

Reunión de los miembros del equipo para puesta en común acerca de todos los avances del seguimiento	24/4	2
Desarrollo de documento y transofmración a latex para el segundo seguimiento	26/4	2
Seguimiento 4		6
Continuar trabajando sobre la documentación y memoria final del proyecto	29/4	2
Primeros bosquejos de la exposición final	30/4	1
Apoyo para solucionar la eficacia a nivel de código	1/5	2
Desarrollo de documento y transofmración a latex para el segundo seguimiento	3/5	1
Semana Final		6
Desarrollo de memoria/documentacion	6/5 - 8/5	3
Desarrollo de presentacion para exposición final	9/5 - 10/5	3
Total		52

9.2. Guillermo Golpe

Actividad Realizada	Fecha	Tiempo (hs)
Seguimiento 1		8
Decisión del tema	11/3	3
Primera aproximación para entendimiento del método SIFT	14/3	2
Puesta en común de la información recaudada para metodologías	15/3	1
Desarrollo de descripción del primer seguimiento	16/3	2
Seguimiento 2		19
Búsqueda de solución debido al patentado de SURF	23/3	3

Obtención de billetes en condiciones	24/3	1
Desarrollo (pair-programming) de creación de referencias en billetes	24/3	3
Generación de imágenes para la base de datos	26/3	3
Testeo con imágenes para primer aproximación de calidad de detectores	30/3	1
Optimización del código funcional	1/4	3
Puesta en común del equipo respecto a avances y trabajo hecho	2/4	3
Creación del documento de segundo seguimiento	4/4	2
Seguimiento 3		11
Sesión ágil de testeo	20/4	2
Reunión de puesta a punto con el equipo para comentar avances y problemas en el seguimiento	24/4	2
Intento básico de aplicar alguna solución a los problemas encontrados	25/4	3
Agregado de información a la documentación del proyecto	25/4	1
Creación del documento correspondiente al seguimiento	26/4	2
Subida de las imágenes testeadas al repositorio de forma ordenada	26/4	1
Seguimiento 4		7
Desarrollo casi completo de solución por cuadrantes	1/5	4
Últimos detalles y correcciones del modelo de cuadrantes	3/5	1
Corrección parcial de la memoria con los nuevos enfoques	3/5	1
Creación del documento correspondiente al seguimiento	4/5	1
Semana Final		8

Escritura de documentación, específicamente centrada en Experimentación, Autoevaluación y verificación de correlación con el resto de partes.	7/5 - 9/5	3
Escritura de conclusión y revisión general de la documentación	9/5	2
Creación de la presentación plasmando lo documentado	10/5	3
Total		53

9.3. Federico Vareika

Actividad Realizada	Fecha	Tiempo (hs)
Seguimiento 1		9
Decisión del tema	11/3	3
Investigación de el funcionamiento del método <i>SURF</i> y revisión de la implementación de repositorio existente	12/3	4
Puesta en común de la información recaudada para metodologías	15/3	1
Transcripción de bosquejos a informe Latex para primer seguimiento	16/3	1
Seguimiento 2		19
Búsqueda de solución debido al patentado de <i>SURF</i>	23/3	3
Intentar de utilizar documentos del gobierno para las imagenes de los billetes en condiciones	24/3	1
Obtención de billetes de coleccion para la base de datos	24/3	1
Desarrollo (pair-programming) de creación de referencias en billetes	24/3	3
Desarrollo de extracción de características <i>SIFT</i> en base a las referencias creadas	24/3	2
Desarrollo del algoritmo de matching con <i>SIFT</i> básico	25/3	3

Puesta en común del equipo respecto a avances y trabajo hecho	02/4	3
Escribir documento del segundo seguimiento	07/4	3
Seguimiento 3		11
Análisis y explicación de los parametros usados para <i>SIFT</i>	22/4	2
Reunión de los miembros del equipo para puesta en común acerca de los avances para el seguimiento	24/4	2
Prueba y análisis de errores de las nuevas imágenes de prueba	25/4	4
Documentación del seguimiento 3	27/4	3
Seguimiento 4		7
Desarrollo de umbrales automáticos	28/4	2
Desarrollo de generación artificial de imágenes	28/4	1
Cambio de uso de regiones a billetes enteros	29/4	1
Cambio de uso de regiones a billetes enteros	1/5	0.5
Limpieza de código, agregar <i>type hinting</i> y unificar <i>SIFT</i> y <i>ORB</i>	3/5	1.5
Documentación del seguimiento 4	3/5	1
Semana Final		8
Desarrollo de memoria/documentacion	6/5 - 9/5	7.5
Desarrollo de presentacion para exposición final	10/5	0.5
Total		54

Referencias

- [1] Introduction to sift (scale-invariant feature transform). https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html. Accessed: 2026-05-07.
- [2] Matthew Brown and David G. Lowe. Invariant features from interest point groups. 2002.
- [3] Faiz M. Hasanuzzaman, Xiaodong Yang, and YingLi Tian. Robust and effective component-based banknote recognition for the blind. *IEEE Transactions on Systems, Man, and Cybernetics Part C: Applications and Reviews*, 42(6):1021–1030, November 2012.
- [4] David G. Lowe. Distinctive image features from scale-invariant keypoints. *Int. J. Comput. Vis.*, 60(2):91–110, 2004.
- [5] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary R. Bradski. Orb: An efficient alternative to sift or surf. In Dimitris N. Metaxas, Long Quan, Alberto Sanfeliu, and Luc Van Gool, editors, *ICCV*, pages 2564–2571. IEEE Computer Society, 2011.